

Yes. I still have visibility over the project context (including earlier parts of this project), and there were quite a few design decisions and technical difficulties that shaped the current architecture. Below is a chronological summary of the significant ones.

1. Overall Architecture

Initial uncertainty

- Composition represented directly vs introducing a `CompositionProject`.
- Number of ViewModels required.
- Where canvas settings should live.

Final decision

- `CompositionProject`
 - owns canvas settings
 - owns layer collection
 - `CompositionViewModel`
 - controls the editor
 - Models remain data only.
-

2. Navigation

Initially experimented with:

- `NavigationService`
- `Pages`

Problems:

- unnecessary complexity
- difficult ViewModel ownership

Final solution:

MainWindow hosts UserControls and simply swaps the current View.

3. MVVM Separation

Several iterations were needed to decide:

Should layers expose Models?

Should ViewModels duplicate collections?

Final architecture:

```
Model
  ↑
ViewModel
  ↑
View
```



Each LayerViewModel owns exactly one Layer model.

No duplicated collections.

4. Canvas Representation

Several possibilities considered:

- Canvas manually filled
- DrawingVisual
- Custom control
- ItemsControl

Final solution:

```
ItemsControl
+
Canvas
```



which naturally fits MVVM.

5. Layer Selection

Problem:

Selecting overlapping layers selected multiple objects.

Solution:

- iterate layers from top to bottom
- stop at first hit

Rendering order and selection order became consistent.

6. Layer Z-order

Question:

How should selected layers be brought to front?

Final decision:

Move the selected layer to the end of the collection.

Rendering naturally follows collection order.

7. Scaling

Initially unclear whether scale should modify

- width
- height
- transform
- bitmap

Final decision:

Store

Scale



independently.

Bitmap never changes.

Rendering applies scaling through transforms.

8. Rotation

Several issues appeared.

Initially

- wrong pivot
- offsets after rotation
- incorrect selection rectangle

Eventually:

- rotation around CenterX/CenterY

- scaling around same center
 - rotation stored in degrees
 - only multiples of 90°
-

9. Selection Rectangle

Initially the rectangle rotated incorrectly.

Problem:

Selection rectangle inherited transforms.

Eventually:

- content rotates
- selection rectangle stays axis-aligned
- bounding box recalculated

This simplified interaction considerably.

10. Text Rendering

Originally:

WPF TextBlock size was reused.

Problem:

Rendered text became clipped.

Reason:

Skia metrics differ from WPF metrics.

Solution:

Measure directly using

```
SKFont.MeasureText()
```



before allocating the bitmap.

11. Text Quality

Initially imported text looked rough.

Reason:

Conversion to 1 bpp destroyed Skia antialiasing.

Solution:

Keep imported compositions grayscale.

Threshold only when producing masks.

12. Importing Composition

Initially:

```
Load
↓
ConvertTo1bpp
↓
Crop
```



Problem:

Destroyed grayscale information.

Final pipeline:

```
Load
↓
Grayscale
↓
Find bounding box
↓
Crop
↓
Store grayscale
```



Huge improvement.

13. Bounding Box Detection

Originally

```
Threshold()
```



was used.

Eventually discovered

```
Compare(gray, Black)
```



works equally well.

Simpler.

14. Preview Generation

Question:

Who generates previews?

Initially there was discussion about

```
CreatePreview()
```



Eventually:

ImageService owns preview generation.

No image processing inside models.

15. Ownership of Images

Question:

Should models convert images?

Final rule:

Only ImageService performs

- loading
- conversion
- rendering
- preview generation

Models only store Mats.

16. Mat Lifetime

Several native memory leaks were identified.

Especially:

```
Mat affineMatrix  
Mat transformed  
Mat mask
```



Eventually standardized on

```
using Mat ...
```



unless ownership is transferred.

17. Reference Semantics

Discussion about

```
ImageLayer layer
```



vs

```
ref ImageLayer layer
```



Clarified that

```
layer.SetBitmap(...)
```



already mutates the object.

No need for `ref`.

18. Save / Load Paths

Question:

How to create project directories?

Clarified distinction between

- filename
- directory
- full path

using

```
SaveFileDialog
```



19. Imported Composition Transparency

Initially preview behaved differently from rendering.

Preview:

correct

Renderer:

always treated white as transparent.

Bug discovered.

Renderer updated to respect

```
TransparentWhite
```



for both

- ImageLayer
- CompositionLayer

Now preview and render behave identically.

20. Gray + Alpha Discussion

Unexpected discovery.

OpenCV supports

```
Gray
```

```
BGR
```

```
BGRA
```



but not

Gray + Alpha



Discussion led to realizing OpenCV's preferred model is

Image

+

Mask



rather than embedded alpha.

This influenced the renderer's implementation.

21. Current Rendering Pipeline

The rendering pipeline gradually converged into a very clean model:

Layer

↓

WarpAffine

↓

Transparent?

↓

Mask?

↓

CopyTo (rendered)



Every layer type now follows the same conceptual workflow.

In hindsight

The biggest difficulties were not coding problems—they were **architecture and semantics**:

- deciding ownership of image processing,
- keeping MVVM clean,
- defining transform semantics,
- preserving image quality,
- making preview and render behave identically,
- avoiding unnecessary bitmap regeneration,
- and arriving at a renderer whose behavior is predictable regardless of layer type.

Those decisions form the foundation of the current implementation. Most remaining work is feature development rather than resolving architectural uncertainty.